ADC en RP2040 y RP2350

Lectura de luz con LDR · acceso por registros y Pico SDK

Equipo 1 · Periférico ADC · Actividad integradora

Integrantes: [Completar]

Raspberry Pi Pico · Raspberry Pi Pico 2 W

Ruta de la presentación

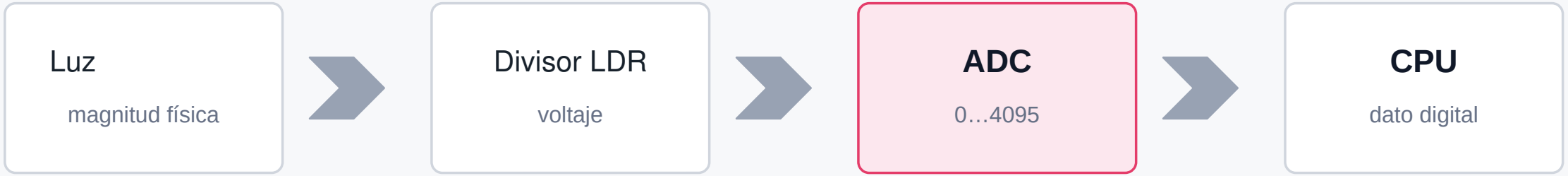
ESTRUCTURA

Estructura y tiempos sugeridos de la actividad

1	Introducción y problema tecnológico	5 min
2	Antecedentes, evolución, aplicaciones y arquitectura	10 min
3	Registros y acceso mediante punteros	7 min
4	Configuración mediante Pico SDK	5 min
5	Comparación RP2040 vs RP2350	5 min
6	Comando REPL	5 min
7	Demostración física	5–8 min

¿Qué problema resuelve un ADC?

El mundo físico es analógico; la CPU trabaja con números.



Aplicación de este proyecto

Divisor LDR → GP26/ADC0 → ADC de 12 bits → voltaje y escala ADC.

Parte 1 · 5 min

ADC del RP2040 y RP2350

Características relevantes para la práctica

Arquitectura

SAR (aproximaciones sucesivas). Convierte una tensión analógica en un código digital.

Resolución

12 bits → 4096 códigos posibles. RP2040: 8.7 ENOB; RP2350: 9.2 ENOB.

Velocidad

Hasta 500 kS/s con reloj independiente de 48 MHz.

Canales

Pico/Pico 2 W: ADC0–ADC3 sobre GPIO26–GPIO29 y sensor interno.

Transferencia

FIFO de 8 muestras, interrupción y DREQ para DMA.

Control

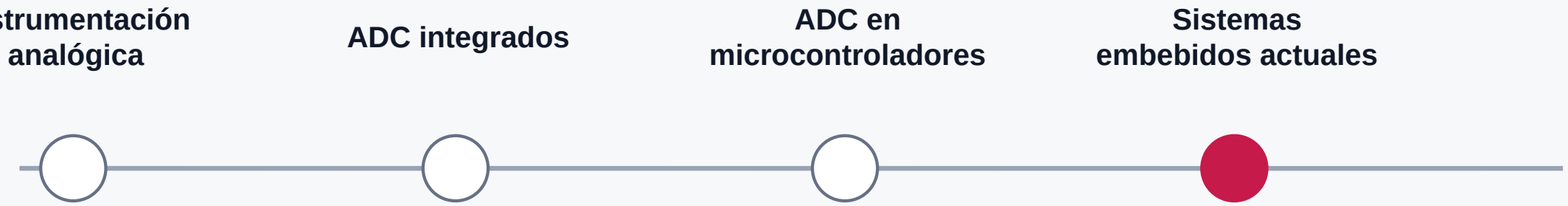
Registros mapeados en memoria o funciones del Pico SDK.

Fuente técnica: Raspberry Pi Pico SDK · hardware_adc

Parte 2 · 10 min

Antecedentes y evolución tecnológica

HISTORIA



Mejoras clave: integración · velocidad · precisión · consumo · flexibilidad

Aplicaciones actuales del ADC

Sensores

Temperatura, luz, presión, fuerza y posición.

Energía

Monitoreo de baterías, fuentes y voltajes.

Control

Realimentación y adquisición de datos en sistemas embebidos.

Instrumentación

Medición y digitalización de señales analógicas.

Caso del proyecto: divisor LDR para demostrar una entrada analógica real con un montaje **mínimo.**

Sensor seleccionado: LDR + 10 kΩ

Resistencia sensible a la luz
Más luz reduce R_LDR. La LDR no genera voltaje por sí sola.

Divisor a 3.3 V
Vnodo = V3V3 × 10000 / (R_LDR + 10000)

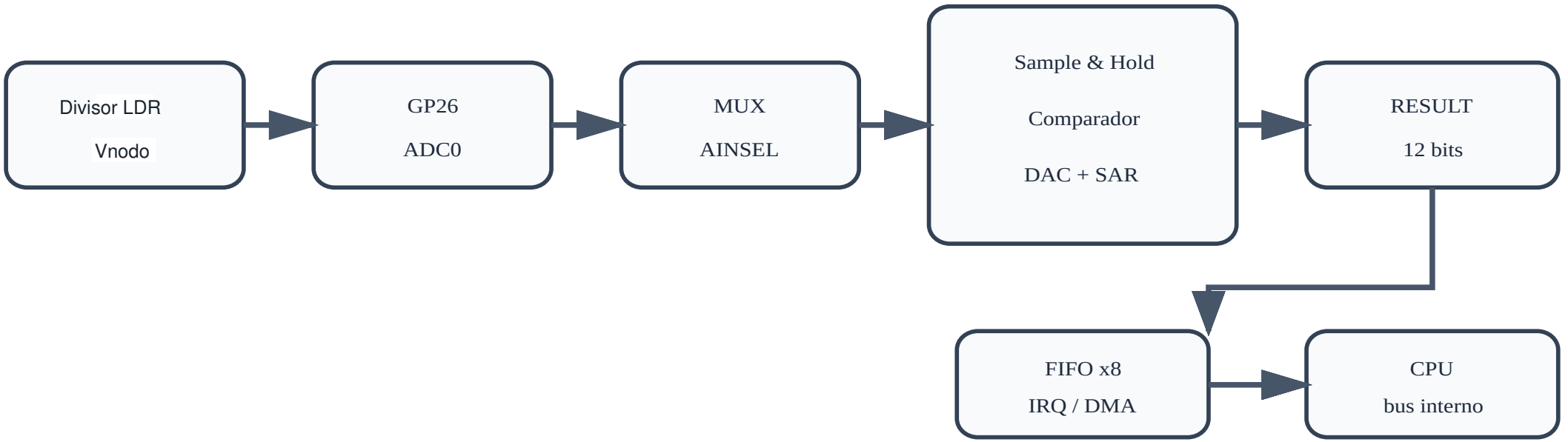
Ejemplo teórico
R_LDR = 10 kΩ → Vnodo = 1.65 V.
No es una medición física.

Conexión
3V3 → LDR → nodo GP26/ADC0 → 10 kΩ → GND

Respuesta
Más luz → mayor voltaje y código ADC.
Al cubrirla, ambos bajan.

Alcance
Voltaje y escala ADC. No lux: hacen falta modelo y calibración.

Arquitectura funcional del ADC



Proyecto: conversión one-shot. FIFO, IRQ y DMA no se usan en esta demo de luz.

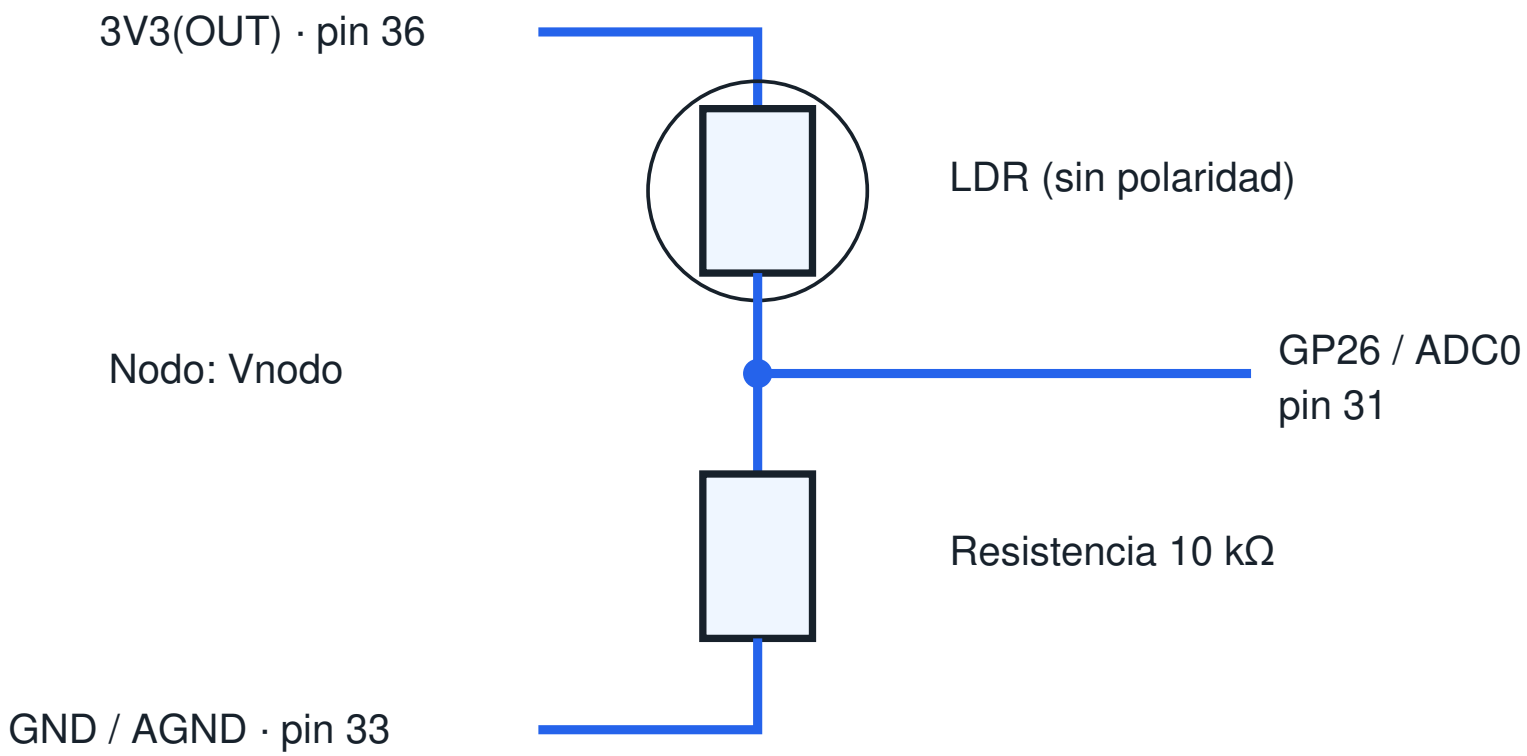
Opciones de configuración del periférico

Opción	Registro/campo	Uso
Canal de entrada	CS.AINSEL	Selecciona ADC0…ADC3 o sensor interno.
Una conversión	CS.START_ONCE	Lectura bajo demanda del divisor LDR.
Conversión continua	CS.START_MANY	Muestreo continuo.
Round-robin	CS.RROBIN	Alterna automáticamente entre canales.
Velocidad	DIV	Divide el reloj para fijar la cadencia de muestras.
FIFO	FCS / FIFO	Almacena hasta 8 resultados; útil con IRQ/DMA.
Interrupción	INTE / INTS	Atiende la FIFO sin sondeo constante.

Configuración elegida: ADC0 + START_ONCE + promedio de 16 muestras; FIFO/IRQ/DMA desactivados.

Diagrama de conexión

Divisor LDR + 10 kΩ · Pico / Pico 2 W



Más luz → menor R_LDR → mayor Vnodo → mayor código ADC

Alimentar con 3V3; no usar VBUS/5 V. GP26 mide el nodo, no va en serie.

Mapa de registros: direcciones base

Bloque	RP2040	RP2350	Uso en la práctica
RESETS	0x4000C000	0x40020000	Sacar el ADC de reset.
IO_BANK0	0x40014000	0x40028000	GPIO26_CTRL.FUNCSEL = NULL.
PADS_BANK0	0x4001C000	0x40038000	Desactivar receptor/pulls; ISO en RP2350.
ADC	0x4004C000	0x400A0000	CS, RESULT y opciones del ADC.

```
dirección del registro = BASE + OFFSET

Ej. RP2040: ADC.CS = 0x4004C000 + 0x000
Ej. RP2350: ADC.RESULT = 0x400A0000 + 0x004
```

Fuentes: datasheets RP2040/RP2350 y headers hardware/regs del Pico SDK

Parte 3 · 7 min

Registros y campos utilizados

Registro	Offset	Campo	Valor / función
RESETS.RESET	0x000	ADC bit 0	Pulso de reset del periférico.
RESETS.RESET_DONE	0x008	ADC bit 0	Esperar 1: ADC disponible.
IO_BANK0.GPIO26_CTRL	0x0D4	FUNCSEL [4:0]	31 (NULL): desconecta función digital.
PADS_BANK0.GPIO26	0x06C	OD/IE/PUE/PDE	1 / 0 / 0 / 0.
PADS_BANK0.GPIO26	0x06C	ISO bit 8	Solo RP2350: 0.
ADC.CS	0x000	EN / START_ONCE / READY	1 / pulso / esperar 1.
ADC.CS	0x000	AINSEL	0 → ADC0 / GP26.
ADC.RESULT	0x004	[11:0]	Resultado 0…4095.

Acceso directo mediante punteros

Dirección → puntero → desreferenciación → hardware

```
static volatile uint32_t *const adc_cs =
    (volatile uint32_t *)(uintptr_t)
    (ADC_BASE_ADDR + ADC_CS_OFS);

*adc_cs |= ADC_CS_START_ONCE;
while (((*adc_cs) & ADC_CS_READY) == 0u) { }
uint16_t raw = (*adc_result) & 0x0FFFu;
```

volatile

Obliga a realizar las lecturas/escrituras porque el hardware puede cambiar el valor.

uint32_t

Los registros accedidos son de 32 bits aunque RESULT use 12 bits.

Máscaras

Permiten modificar o consultar solo los bits necesarios.

Secuencia exigida: BASE → offset → dirección → puntero → valor → periférico

Inicialización por registros

Qué ocurre antes de leer el divisor LDR

1	**RESETS**	Reset y liberación del ADC.
2	**IO_BANK0**	GP26 FUNCSEL = NULL.
3	**PADS_BANK0**	OD=1, IE=0, pulls=0; ISO=0 en RP2350.
4	**ADC.CS**	EN=1 y esperar READY.
5	**AINSEL**	Canal 0 → GP26.
6	**Conversión**	START_ONCE → READY → RESULT.

Configuración equivalente con Pico SDK


```
adc_init();
adc_gpio_init(26);
adc_select_input(0);
uint16_t raw = adc_read();
```

Función SDK	Qué abstrae
adc_init()	RESETS, CS.EN y espera READY.
adc_gpio_init(26)	FUNCSEL=NULL, pulls e input digital del pad.
adc_select_input(0)	Campo CS.AINSEL.
adc_read()	START_ONCE, READY y RESULT.

Ventaja

Código corto y portable entre RP2040 y RP2350.

Limitación

Oculta direcciones y campos; para comprender el periférico hay que consultar el mapa real.

Parte 4 · 5 min

Registros vs SDK: misma función, distinta abstracción

Acceso directo

- Direcciones base escritas por plataforma.
- Punteros volatile.
- Máscaras y campos explícitos.
- Hay que manejar ISO y cambios de mapa.

Mayor control y mejor visibilidad del hardware.

Pico SDK

- Misma API en las dos tarjetas.
- Headers correctos seleccionados al compilar.
- Configuración de GPIO y ADC encapsulada.
- Menor riesgo de errores de dirección.

Más legibilidad y portabilidad.

El comando REPL permite ejecutar ambos backends sin cambiar el circuito.

Comparación técnica: RP2040 vs RP2350

Elemento	RP2040	RP2350A / Pico 2 W
ADC	SAR · 12 bits · 500 kS/s	SAR · 12 bits · 500 kS/s
ENOB	8.7	9.2
ADC_BASE	0x4004C000	0x400A0000
RESETS_BASE	0x4000C000	0x40020000
IO_BANK0_BASE	0x40014000	0x40028000
PADS_BANK0_BASE	0x4001C000	0x40038000
CS.AINSEL	bits [14:12]	bits [15:12]
CS.RROBIN	bits [20:16]	bits [24:16]
Pad ISO	No existe	Sí · debe quedar en 0
Código SDK	Mismas llamadas	Mismas llamadas

Parte 5 · 5 min

Comando integrado al REPL

luz <regs|sdk>
16 muestras por software · ADC bruto · voltaje · escala ADC (no lux)

Ejemplo numérico ilustrativo

```
RP2350> luz regs
argv[1]     : regs
ADC bruto   : 2048 / 4095
Voltaje     : 1.650 V
Escala ADC : 50.0 % (no lux)
```

V = raw × 3.3 / 4096
Escala ADC = raw × 100 / 4095

El porcentaje normaliza el código; no es porcentaje físico de iluminación.

Validación
`luz otra` → backend inválido.
`luz` → mostrar uso.

Validación de argumentos con argc y argv


```
if (argc != 2) {
    printf("Uso: luz <regs|sdk>\n");
    return -1;
}

if (strcmp(argv[1], "regs") == 0) {
    backend = &adc_backend_regs;
} else if (strcmp(argv[1], "sdk") == 0) {
    backend = &adc_backend_sdk;
} else {
    printf("Error: backend invalido\n");
    return -1;
}
```

argv[0]

Contiene el nombre del comando: luz.

argv[1]

Selecciona la implementación: regs o sdk.

Resultado observable

La luz cambia el voltaje del divisor.
Se muestran voltaje y escala ADC (no lux).

Demostración física · pruebas pendientes

1. Conectar: 3V3 → LDR → nodo GP26 → 10 kΩ → GND.
2. En RP2040: cargar firmware; ejecutar **luz regs** y **luz sdk**.
3. Cubrir la LDR y repetir: se espera que ADC y voltaje bajen.
4. Iluminarla y repetir: se espera que suban.
5. Repetir en Pico 2 W / RP2350 con el mismo divisor.
6. Comparar con luz estable y guardar solo evidencias reales.

No se exigen códigos fijos ni igualdad exacta entre muestras.

Conclusiones

- El divisor LDR + 10 kΩ convierte cambios de resistencia en voltaje; el ADC digitaliza ese voltaje.
- Se muestran voltaje estimado y escala ADC; no se calculan lux.
- Los punteros exponen direcciones, offsets, volatile, máscaras y campos reales.
- Pico SDK realiza la misma lectura mediante una API común.
- RP2040 y RP2350 conservan la lógica del ADC, con diferencias en direcciones y campos.

La validación física debe documentarse después de las pruebas.

Referencias técnicas

- Raspberry Pi Ltd. · RP2040 Datasheet.
- Raspberry Pi Ltd. · RP2350 Datasheet.
- Raspberry Pi Ltd. · Raspberry Pi Pico 2 W Datasheet.
- Raspberry Pi Pico SDK · hardware_adc y headers hardware/regs.
- Adafruit · Photocells / Using a Photocell.
  learn.adafruit.com/photocells/using-a-photocell
  Principio del divisor; montaje adaptado a 3.3 V.

Documentación: README + docs/ + diagramas/ + firmware/ + guion